StyleCart Project Report

Full-Stack Architecture, Feature Walkthrough, and Interview Study Guide

Prepared: 2026-10-05 **Project:** StyleCart fashion e-commerce platform **Audience:** Java full-stack interviews, project walkthroughs, and onboarding

> *This report is based on the source tree in this workspace. It separates implemented behavior from README claims and inferred technology rationale. The application is a portfolio/demo system, not a production payment or security reference implementation.*

1. Executive Summary

StyleCart is a fashion storefront with customer and administrator experiences. Its frontend is a React and TypeScript single-page application. Its backend is a Java 17 / Spring Boot REST API organized into controllers, services, repositories, JPA entities, DTOs, and security configuration. The intended persistent store is MySQL, with Docker Compose describing MySQL, the backend, and a Vite frontend.

The frontend includes catalog browsing, filtering, product detail, a cart, wishlist, checkout, order history, profiles, and administrative screens. The backend has API surfaces for authentication, catalog, categories, carts, wishlists, orders, profiles, and administration. The schema models users, categories, products, cart and wishlist lines, orders, and order lines.

An important implementation fact is that this is currently a hybrid demo. Frontend services call the REST API when `VITE_API_URL` is set, but many catch API errors and fall back to a localStorage simulation. The simulated repository is initialized at frontend startup and is not a synchronized offline cache. Therefore a screen can appear to work while the backend call failed, and local demo data can diverge from MySQL.

Interview-sized project description

> *"StyleCart is a full-stack apparel commerce demo. The UI is a React/TypeScript SPA organized around reusable pages, contexts, and service modules. The Java 17 Spring Boot backend exposes REST endpoints and uses Spring Security with signed JWTs, Spring Data JPA, and MySQL. The domain covers catalog, variants, carts, wishlists, checkout, order fulfillment, and admin management. A localStorage adapter supports preview/demo behavior. The most important next step before production is to make backend integration the explicit source of truth, harden authorization and secrets, and replace simulated payment with server-verified gateway transactions."*

2. Architecture at a Glance

```
Browser
  React pages and components
       | state via Auth / Cart / Wishlist contexts
       v
  frontend service modules (Axios + local simulation fallback)
       | HTTP JSON, Authorization: Bearer <JWT>
       v
Spring Boot REST API (:8080, /api/...)
  Controllers -> Services -> Spring Data Repositories
       |                            |
       | domain rules               v
       |                        JPA / Hibernate
       |                            |
       +--------------------------> MySQL 8

Alternative frontend-only demo path:
  service modules -> simDb -> browser localStorage

Payment demo path:
  CheckoutPage -> Razorpay browser SDK -> frontend order service
  Backend OrderService currently marks non-COD orders paid itself;
  it does not verify a Razorpay signature or use PaymentService.
```

Main request lifecycle

- A page reads data through a frontend service such as `productService` or `cartService`.

- If `VITE_API_URL` is truthy, the service tries Axios. The shared client adds the stored JWT bearer token.
- On many failures, that service logs a warning and returns data from the localStorage simulation instead.
- For a real API request, Spring Security checks the request. The JWT filter validates the token and loads the user and authorities.
- A controller binds the URL and request DTO, then delegates business work to a service.
- The service applies rules and calls Spring Data repositories. JPA maps entities to MySQL rows and relationships.
- The service maps entities into response DTOs; the controller returns JSON and an HTTP status.

3. Technology Stack and Why It Fits

The rationale below is engineering rationale inferred from the selected technologies and code structure; it should not be presented as a documented author decision unless the team confirms it.

Backend

- **Java 17:** LTS-era language/runtime, mature ecosystem, static typing, and a common interview stack for enterprise API development.
- **Spring Boot 3.3 / Spring Web:** convention-based application setup and REST controllers, dependency injection, configuration, and HTTP integration with less manual wiring.
- **Spring Security 6:** centralized authentication and route authorization, rather than duplicating access checks in each controller.
- **JWT with JJWT 0.12.5:** signed, stateless bearer authentication suitable for JSON APIs. The code uses HMAC signing, a 24-hour configured expiry, and BCrypt password encoding.
- **Spring Data JPA / Hibernate:** entity mapping and repository abstractions reduce repetitive SQL for common reads and writes. `@Transactional` provides a boundary for multi-row business operations.
- **MySQL 8 / InnoDB:** relational constraints and transactions fit commerce entities with strong references among users, products, carts, and orders.
- **Jakarta Bean Validation:** request DTO constraints reject malformed input at the HTTP boundary.
- **Maven:** conventional Java dependency and build management; the Docker build uses Maven to create the Spring Boot executable JAR.
- **SpringDoc OpenAPI and Actuator:** generated API documentation and health/metrics endpoints aid development and operations.
- **H2 dependency:** available at runtime for a potential lightweight test/development database; current configuration shown is MySQL, and no automated tests were found in the workspace scan.

Frontend

- **React 19:** component-based UI composition and local state for screens and interactions.
- **TypeScript:** compile-time contracts for products, addresses, carts, orders, roles, and API-facing data shapes.
- **Vite:** fast local development server and frontend bundler. The root package defines `npm run dev` on port 3000 and `npm run build`.
- **Tailwind CSS 4:** utility-first styling used directly in component markup, with project fonts defined in `src/index.css`.
- **Axios:** centralized HTTP client with a timeout and request interceptor for bearer-token injection.
- **Lucide React:** consistent UI icons. **Canvas Confetti** is used for the checkout success celebration. `motion` is in dependencies, but its presence alone does not prove it is used in the main UI.
- **React Context:** a small global state mechanism for authentication, cart, and wishlist state without adding a dedicated state library.

Data and delivery

- **MySQL schema script:** initial DDL and demo seed data in `database/schema.sql`.
- **Docker Compose:** wires a MySQL service, a backend image build, and a Node/Vite frontend service for local multi-service startup.
- **Multi-stage Dockerfile:** builds with Maven plus a JDK image, then copies only the executable JAR into a smaller JRE runtime image; the runtime process uses a non-root account.

4. Repository Map: Important Files and Responsibilities

Root and frontend entry points

- `package.json`: authoritative root frontend scripts and dependencies for the `src/` tree. The project name is still `react-example`, and some dependencies may be leftover scaffolding.
- `index.html`: Vite HTML entry page.
- `vite.config.ts`: React and Tailwind Vite plugins, `@` path alias, and HMR/watch settings. It does **not** define a `/api` development proxy.
- `tsconfig.json`: TypeScript compiler configuration.
- `src/main.tsx`: mounts `<App />` into the `#root` element and loads global CSS.
- `src/App.tsx`: composition root. It installs the three contexts, keeps a string-based `currentView`, renders storefront or admin layouts, and coordinates navigation and selected product/order state. It is not using React Router, so views are not naturally represented as browser routes.
- `src/index.css`: imports Tailwind and defines the serif, sans-serif, and monospace font tokens.
- `src/types/index.ts`: shared TypeScript domain contracts including `Product`, `Cart`, `Order`, `Address`, roles, statuses, and dashboard stats.
- `src/data/initialData.ts`: local simulation seed data used by the frontend demo.

Frontend state and service boundary

- `src/context/AuthContext.tsx`: holds current user and auth-modal state; delegates login/register/logout to `authService`. If no stored user exists, it automatically creates a demo customer session using a simulated token.
- `src/context/CartContext.tsx`: loads cart state, exposes add/update/remove/clear/coupon actions, and opens the cart drawer after adding an item.
- `src/context/WishlistContext.tsx`: loads and refreshes wishlist state, toggles items, and coordinates moving items into the cart.
- `src/services/api.ts`: configures Axios, injects a bearer token from localStorage, initializes the simulated database, and implements localStorage persistence/recalculation helpers. `BACKEND_MODE` is declared but not used as a mode selector in the code searched.
- `src/services/productService.ts`: catalog list/filter/sort/page operations and product CRUD. It tries the API if `VITE_API_URL` exists, then falls back to the local simulated catalog.
- `src/services/authService.ts`: API-first login/registration with a local fallback; stores token and user in localStorage. The simulated token is not a server-issued JWT.
- `src/services/cartService.ts`: API-first cart operations and local simulation for cart line matching, totals, delivery threshold, and coupon codes.
- `src/services/wishlistService.ts`: fetches and mutates wishlist state; current implementation also directly changes local simulated data even when attempting API requests.
- `src/services/orderService.ts`: API-first order actions, with a local fallback that creates order numbers, deducts/restores simulated stock, and updates simulated statuses.
- `src/services/userService.ts`: current user profile operations.
- `src/services/adminService.ts`: dashboard aggregates and category/user administration, including local fallback and demo monthly revenue series.

- `src/services/razorpay.ts`: dynamically loads Razorpay Checkout, chooses a browser-side key, and opens its popup. It is a client SDK wrapper, not payment verification.

Customer UI

- `src/pages/HomePage.tsx`: storefront landing/catalog entry and featured selections.
- `src/pages/ProductsPage.tsx`: catalog result view receiving category, sort, price, and keyword parameters from `App.tsx`.
- `src/components/product/ProductGrid.tsx`, `ProductCard.tsx`, and `FilterSidebar.tsx`: product listing composition, reusable product display, and filter controls.
- `src/pages/ProductDetailPage.tsx`: product inspection, variant selection, and purchase actions.
- `src/components/common/SearchModal.tsx`: search interaction and navigation to results.
- `src/pages/CartPage.tsx` and `src/components/cart/CartDrawer.tsx`: full cart and quick slide-over interactions.
- `src/pages/WishlistPage.tsx`: saved products and move-to-cart workflow.
- `src/pages/CheckoutPage.tsx`: address validation, payment choice, Razorpay popup, order submission, and success navigation.
- `src/pages/OrderSuccessPage.tsx`, `OrdersPage.tsx`, and `OrderDetailPage.tsx`: confirmation, order list/detail, and cancellation/tracking views.
- `src/components/order/OrderTimeline.tsx` and `OrderStatusBadge.tsx`: reusable status presentation.
- `src/pages/ProfilePage.tsx`, `AboutPage.tsx`, and `ContactPage.tsx`: account and informational views.
- `src/pages/AuthModal.tsx`: login/register modal controlled by `AuthContext`.
- `src/components/layout/Navbar.tsx` and `Footer.tsx`: primary storefront navigation and shared footer.
- `src/components/common/SizeGuideModal.tsx`: product size guidance modal.

Administration UI

- `src/components/admin/AdminSidebar.tsx`: admin navigation.
- `src/pages/admin/AdminDashboardPage.tsx`: KPI/revenue dashboard.
- `AdminProductsPage.tsx` and `AdminProductFormPage.tsx`: product listing and create/edit form.
- `AdminCategoriesPage.tsx`: category management.
- `AdminInventoryPage.tsx`: stock monitoring and adjustments.
- `AdminOrdersPage.tsx`: order fulfillment/status updates.
- `AdminUsersPage.tsx`: customer administration.

Backend

- `backend/pom.xml`: Java 17 / Spring Boot 3.3 dependency versions and Maven build plugins.
- `backend/src/main/java/com/stylecart/StyleCartApplication.java`: Spring Boot application entry point and component scan root.
- `controller/`: HTTP routes and request-to-service delegation. Controllers include auth, products, categories, cart, wishlist, orders, users, and admin.
- `service/`: core business logic. `OrderService` is especially important because it checks cart contents, checks/deducts stock, calculates totals, creates order lines, clears the cart, and restores stock on eligible cancellation.
- `repository/`: Spring Data interfaces for persistence and query definitions.
- `entity/`: JPA mapping classes and enums for users, roles, products, categories, carts, cart items, wishlists, orders, order items, address, and statuses.
- `dto/`: request/response contracts separating external JSON shapes from entities.

- `security/JwtService.java`: signs and parses JWTs using configured Base64 secret, subject email, user ID, role, and expiration.
- `security/JwtAuthenticationFilter.java`: reads `Authorization: Bearer ...`, validates token, loads the user, and sets Spring's `SecurityContext` for the request.
- `security/UserDetailsServiceImpl.java` and `UserPrincipal.java`: connect persisted users and roles to Spring Security's identity model.
- `config/SecurityConfig.java`: stateless filter chain, BCrypt password encoder, public endpoint rules, and admin/customer restrictions.
- `config/CorsConfig.java`: CORS configuration. Its current wildcard-origin pattern with credentials is too permissive for deployment.
- `config/OpenApiConfig.java`: API documentation metadata/configuration.
- `exception/GlobalExceptionHandler.java`: translates known missing-resource, bad-request, validation, credentials, access-denied, and generic exceptions into HTTP error responses.
- `backend/src/main/resources/application.yml`: server port, datasource environment variables, JPA behavior, JWT values, upload limits, CORS settings, OpenAPI, and Actuator exposure.

Database and containers

- `database/schema.sql`: creates the MySQL schema, tables, indexes, foreign keys, unique constraints, and demo users/catalog/orders.
- `Dockerfile`: multi-stage backend image build and JRE runtime.
- `docker-compose.yml`: local MySQL/backend/frontend orchestration. The frontend service mounts the repository and runs Vite, so it is a development setup rather than a static production frontend image.
- `frontend/package.json`: a second package manifest exists under `frontend/`, but the source application is at root `src/`, root Vite config points there, and Compose runs `npm install` from `/app` at the repository root. Treat root `package.json` as the active frontend manifest unless deployment scripts say otherwise.
- `README.md`: feature descriptions, setup guidance, endpoint summary, credentials, and interview talking points. Some claims, particularly production readiness and payment integration, should be reconciled with the implementation notes in this report.

5. Domain Model and Persistence

Main relationships

- A **User** has a role and embedded/default address data. A user has one cart and one wishlist in the schema through unique `user_id` values.
- A **Category** groups products. A product belongs to one category.
- A **Product** contains core catalog/price/stock fields. Images, sizes, and colors are JPA element collections stored in separate relational tables.
- A **Cart** belongs to a user and owns cart items. A cart line references a product and stores chosen size, color, and quantity. Cart rows are intended to be mutable and current.
- A **Wishlist** belongs to a user and owns wishlist entries. The database has a unique `(wishlist_id, product_id)` constraint to prevent duplicates.
- An **Order** belongs to a user, embeds the shipping address, and owns order items. Order items retain product name/image/variant/price snapshots while also optionally referencing the product. This preserves what was ordered if catalog data later changes or the product is deleted.
- **OrderStatus** models fulfillment separately from **PaymentStatus**. This separation is important: shipment progress and financial settlement are related but not identical state machines.

Integrity and transaction concepts

`database/schema.sql` uses InnoDB foreign keys and indexes on common lookup fields. Several JPA associations use cascades and orphan removal for child rows such as cart and order lines. Product deactivation is a soft-delete pattern (`active=false`) to retain order history. `OrderService.createOrder` is marked `@Transactional`, so order, stock updates, and cart clearing are intended to succeed or roll back together when a runtime failure occurs.

However, the code does not show pessimistic stock locking or an optimistic `@Version` column. A transaction alone does not prevent two concurrent checkouts from both reading the same stock before decrementing it. Stronger concurrency control is needed for a real store.

There are two schema-management mechanisms in the repository: a handwritten SQL initializer and Hibernate `ddl-auto: update`. A production system should choose a versioned migration strategy such as Flyway or Liquibase rather than rely on schema auto-update.

6. Feature Walkthrough: How the User Journeys Work

A. Browse, search, and inspect products

- `App.tsx` receives navigation actions and stores page inputs such as category, sort, max price, or keyword in React state.
- `ProductsPage` obtains catalog data through `productService`; product listing/filter components render the results.
- In the local simulation, the service filters active products by category, brand, price, rating, size, color, and keyword, sorts them, and slices a page of results.
- In backend mode, the service maps several filters into query parameters for `GET /products`. The backend returns a Spring `Page<ProductDTO>` with pagination metadata.
- Selecting a card stores the selected product in `App.tsx` and switches the view to `product-detail`.
- The detail page lets the shopper choose a size/color and add a line to the cart or save a wishlist item.

The backend catalog supports category, brand, price range, rating, pagination, sorting, keyword search, featured products, new arrivals, brand list, and admin CRUD/stock operations. Note that frontend filters such as size, color, keyword, and UI-specific sort choices are not all sent by the current `getProducts` API call; they are implemented in the local path. This is a concrete contract gap to resolve before claiming feature parity in live mode.

B. Cart and wishlist

- `CartContext` loads cart state once and exposes operations to child components.
- A cart line is distinguished by product plus selected size and color. Adding an identical variant increments quantity; a different variant creates a distinct line.
- Prices are based on the discounted price when present. The UI/local simulator recalculates subtotal, item count, delivery, and total. The current delivery rule is free above $50; otherwise it charges $5.99 (except for an empty cart).
- The coupon helper recognizes `STYLE40` and `WELCOME10` in the local service. There is no corresponding backend coupon endpoint in the inspected API.
- Wishlist actions add/remove products and can transfer a product into the cart using the first available size/color defaults when the user did not choose variants.

The backend validates product activity and available stock when adding/updating a cart item and checks that a cart item belongs to the authenticated user's cart before mutation. The local simulation does not enforce the same full set of invariants in every operation, which is another reason the two modes can behave differently.

C. Authentication and authorization

Backend login path:

- `POST /api/auth/login` binds an email/password DTO.
- `AuthService` asks Spring's `AuthenticationManager` to authenticate.
- `UserDetailsServiceImpl` loads the user; BCrypt compares the supplied password to the stored hash.

- `JwtService` signs a token containing subject/email, user ID, role, issued time, and expiry.
- The frontend stores the token and user response in localStorage; Axios attaches the token to subsequent requests.
- `JwtAuthenticationFilter` validates signed claims and populates the security context for each stateless request.

Backend registration path: the service checks email uniqueness, BCrypt-hashes the password, creates a user, provisions an empty cart and wishlist, and returns a JWT. The current service also honors an optional role sent in the public registration DTO. That is a critical privilege-escalation flaw; registration must always force `ROLE_USER`, and administrative role assignment must be restricted to trusted admin tooling.

Frontend demo behavior: if no user is saved, `AuthContext` automatically sets a simulated default customer. Local authentication checks whether an email exists but does not verify the password against a secure hash. Simulated tokens are not signed by the backend. These conveniences are useful for a preview but are not authentication.

D. Checkout, payment, and order placement

- `CheckoutPage` collects and minimally validates a delivery address, then moves to a review/payment step.
- COD skips the payment popup and calls `orderService.placeOrder`.
- The Razorpay option loads the external browser SDK and opens a checkout popup. The frontend converts the cart total by a hardcoded rate of 84, rounds to a whole rupee, and sends the browser callback fields along with order details.
- On successful callback, the frontend submits the order and navigates to an order-success view; confetti is UI-only feedback.
- In the local simulation, `orderService` checks that the cart is nonempty, deducts simulated inventory, creates an order and order lines, sets payment status to pending for COD or paid otherwise, saves it to localStorage, and clears the cart.
- In backend mode, `POST /api/orders` reads the authenticated user's persisted cart. `OrderService` checks stock, creates order lines and price snapshots, deducts stock, calculates totals, saves the order, and clears the cart in one transaction.

Payment truth: the backend's `OrderService` does not call its injected `PaymentService`. It sets every non-COD order to `PAID` and generates a transaction ID. It does not verify Razorpay's signature or create/verify a gateway order server-side. The browser SDK integration is therefore a demo flow, not secure payment processing. Do not claim production Razorpay verification during an interview.

E. Order history, cancellation, and fulfillment

- Customers can list their own orders, inspect one by order number, and cancel while the order is not shipped, out for delivery, or delivered.
- Cancellation changes the order status and restores each still-linked product's stock.
- Admins can list paginated orders, optionally filter by status, and update fulfillment status/tracking number.
- When a COD order reaches `DELIVERED`, the backend marks its payment status `PAID`.
- The UI has reusable status badges/timeline components for delivery tracking.

There is no explicit backend validation of every allowed transition (for example, ensuring a status only advances along a legal path). A stronger state machine should define permitted transitions and audit actor/time changes.

F. Admin experience

The admin screens provide dashboard metrics, product CRUD, category management, inventory adjustment, order status/tracking updates, and user listing. The backend protects `/api/admin/**` and uses role rules/method checks on protected operations. Dashboard totals are aggregated in `AdminService`; the frontend local fallback uses local demo data and includes a hardcoded monthly revenue series, so that series is not evidence of live analytics.

7. API Surface Summary

All backend controller mappings are rooted at `/api`.

- **Authentication:** `POST /api/auth/login`, `POST /api/auth/register`.

- **Products:** `GET /api/products`, `/search`, `/{id}`, `/featured`, `/new-arrivals`, `/brands`; admin `POST`, `PUT`, `DELETE`, and stock update routes.
- **Categories:** public reads; admin create/update/deactivate operations.
- **Cart:** authenticated get, add line, update quantity, remove line, clear cart.
- **Wishlist:** authenticated list, add/remove, and move-to-cart.
- **Orders:** create, current-user list, detail, cancel; admin list/status update.
- **Users:** authenticated profile get/update; admin user listing.
- **Admin:** dashboard metrics and user listing.
- **Operations/docs:** Actuator endpoints, OpenAPI JSON, Swagger UI.

The complete endpoint contract should be treated as the Java controller source, not the README alone. In particular, verify payload shape and pagination when wiring frontend services to each endpoint.

8. Security, Reliability, and Production Readiness Review

These are source findings to understand and prioritize, not changes made by this report.

Critical: role can be self-assigned at registration

`RegisterRequest` exposes a `role`, and `AuthService.register` uses it when present. Since registration is public, a caller can request an admin role and receive a signed token with that role. Force the role to `ROLE_USER` in the service and remove role from the public registration contract; provide a separately authorized admin workflow for role changes.

High: payment is trusted on the client / backend marks it paid

Never trust a browser callback as proof of payment. Create gateway orders on the server, verify webhook or callback signatures using the secret on the server, make order/payment updates idempotent, and only mark an order paid after verified gateway confirmation. The committed/default browser test key is not a production credential.

High: committed/default secrets and demo credentials

`application.yml`, Compose, and the README contain default JWT/database credentials and demo logins. Rotate secrets, keep them out of source control, inject them through a secret manager/environment, and use distinct production database credentials. Do not expose the seed account passwords publicly.

High: broad CORS and frontend-local token storage

`CorsConfig` allows wildcard origin patterns with credentials. Restrict origins to the actual deployed frontend. LocalStorage bearer tokens are accessible to injected JavaScript in an XSS event; consider a carefully configured HttpOnly/Secure/SameSite cookie design or document the threat tradeoff and harden CSP/input rendering.

High: live API errors silently switch the source of truth

Many frontend services catch any request failure and continue with simulated localStorage. In deployment, this can mask authorization, endpoint, serialization, and network errors; it can also create orders or admin edits that exist only in one browser. Make live/demo mode explicit, avoid fallback for writes, surface API errors, and never label local demo outcomes as persisted server data.

Integration and consistency issues

- The frontend Axios methods request paths such as `/products`, but Java controllers are rooted at `/api/products`. The frontend default base URL `/api` works only when same-origin routing is configured; Vite has no proxy in `vite.config.ts`. Compose sets `VITE_API_URL=http://localhost:8080`, which omits `/api`; requests therefore target paths such as `http://localhost:8080/products` rather than `http://localhost:8080/api/products`. Configure the base URL consistently (for example, include `/api`) or implement a matching proxy.

- The live frontend catalog query omits some filters/sort options supported by the local path; coupons also have only a local implementation.
- Wishlist mutation logic updates localStorage even when API mode is configured and suppresses some API errors.
- Cart totals and order totals are not perfectly symmetric: local cart supports a discount amount/coupons, but backend order creation recalculates from persisted cart and does not apply a coupon field. The API order request does not establish a verified payment contract.
- Prices/delivery thresholds are described in dollars in business logic/UI copy, while checkout uses INR and a hardcoded USD-to-INR conversion. Pick a currency model, store monetary values with explicit currency, and calculate server-side.
- `spring.jpa.hibernate.ddl-auto: update` plus a SQL init script creates multiple schema authorities. Use migrations for controlled deployments.
- The backend Dockerfile is multi-stage, but the Compose frontend is a Vite dev server with the repository bind-mounted and `npm install` at startup. Build and serve immutable static assets behind a production web server/CDN for a production topology.
- The repository contains a second `frontend/package.json`; make the active app root unambiguous and remove/maintain duplicate manifests intentionally.
- A workspace file scan found no test/spec files. Add service tests for auth/order rules, repository integration tests, API/security tests, and frontend workflow tests.

Further engineering improvements

- Add stock locking/version checks to prevent overselling under concurrent checkout.
- Add idempotency keys to order/payment creation and use a unique order number generator with collision handling.
- Enforce order transition rules and record an audit trail.
- Add DTO validation for quantities, address fields, image URLs, prices, roles, and status transitions.
- Return generic server errors to clients while logging detailed exceptions server-side; avoid reflecting arbitrary exception messages.
- Restrict Actuator exposure, enable TLS at the edge, set security headers, rate-limit login, and monitor failed authentication.
- Replace field injection with constructor injection for testability and explicit dependencies.
- Add accessibility checks, responsive UI tests, observability, and structured logs.

9. Running and Understanding the Project

Local frontend

From the repository root, install dependencies and run:

```
npm install
npm run dev
```

The root script serves Vite on port 3000. Without an API base/proxy, the localStorage-backed demo path is the reliable preview path.

Backend

Prerequisites are Java 17+, Maven, and MySQL 8. The README suggests importing `database/schema.sql`, then running `mvn spring-boot:run` from `backend/`. The API server defaults to port 8080. Swagger UI is configured at `/swagger-ui.html`; Actuator includes health, info, and metrics.

Set database values with `MYSQL_HOST`, `MYSQL_PORT`, `MYSQL_DATABASE`, `MYSQL_USER`, `MYSQL_PASSWORD`, and set `APP_JWT_SECRET` in the process environment. The actual YAML references a property named `app.jwt.secret` with a built-in fallback; Spring environment-variable mapping normally uses `APP_JWT_SECRET`.

Docker Compose

`docker-compose up --build` starts MySQL, builds/runs the Java backend, then starts the root Vite app on port 3000. MySQL runs its initialization SQL when its data volume is first created. A persistent volume means editing the SQL seed file does not automatically reseed an already initialized database.

Before relying on Compose for end-to-end API behavior, correct the frontend API base path and replace checked-in default secrets. Compose currently prioritizes a convenient demo over production security.

Build/check commands

- Frontend type check: `npm run lint` (root script runs `tsc --noEmit`).
- Frontend bundle: `npm run build`.
- Backend test/build: from `backend/`, `mvn test` and `mvn package`.
- Automated tests: none were found in the repository scan used to prepare this report; do not imply these commands pass unless run separately.

10. Interview Preparation

A clear 90-second walkthrough

- **Problem:** explain the store's shopper and administrator workflows.
- **Architecture:** browser UI -> frontend service boundary -> REST controller -> service -> repository/JPA -> MySQL.
- **Design choice:** React/TypeScript supports reusable interactive views and typed contracts; Spring Boot provides a conventional layered Java API; relational persistence fits orders/inventory consistency.
- **Core workflow:** checkout uses the persisted user's cart, validates stock, creates order snapshots, decrements inventory, and clears the cart in a transaction.
- **Security:** BCrypt passwords, stateless JWT bearer requests, role-based access rules. Be candid that current role registration and payment behavior need hardening.
- **Tradeoffs/next steps:** explain localStorage preview mode, API fallback risks, payment verification gap, concurrent stock locking, tests, and migrations.

Questions you should be ready to answer

- **Why layers?** Controllers handle HTTP concerns; services contain use cases and transactions; repositories isolate persistence; DTOs prevent direct entity exposure. This improves testability and change isolation.
- **Why JWT and stateless sessions?** The API does not keep per-user HTTP session state, which simplifies horizontal scaling. The tradeoff is token revocation and secure token storage; expiry and refresh/revocation strategy matter.
- **How does a request become authenticated?** Axios sends a bearer token; the filter validates its signature/expiry, extracts the subject, reloads user authorities, then Spring Security applies URL/method authorization.
- **How is stock handled?** The service checks each line and decrements inside an order transaction. Explain the remaining race condition and how row locks or optimistic versioning would prevent overselling.
- **Why DTOs?** They define API contracts, constrain input, and decouple JSON from persistence implementation.
- **What does `@Transactional` guarantee?** Atomic commit/rollback for participating database operations on the transaction; it does not automatically solve concurrency races or external payment side effects.
- **Why snapshot order line details?** Historical orders must preserve purchased name, image, variant, and price when product catalog data changes.
- **How would you test it?** Unit-test service rules with mocked repositories; integration-test JPA/transactions with a test DB; security-test roles/token expiry; API-test validation/status codes; browser-test checkout/cart/admin flows.
- **What would you fix first?** Force user role at registration; make payment server-verified; eliminate silent fallback in live mode; fix `/api` base URL; remove hardcoded secrets and wildcard credentialed CORS; add stock concurrency protection and tests.
- **What is demo-only?** Auto-authentication, fake JWTs, browser localStorage simulation, seeded credentials, hardcoded revenue samples, test payment key, client-side success callback, and simulated backend payment service.

Claims to avoid

- Do not say the current app securely processes real Razorpay transactions.
- Do not say localStorage is the same database as MySQL or is synchronized with it.
- Do not say the current stock decrement is protected from concurrent overselling.
- Do not claim the repository has automated tests without adding/running them.
- Do not describe the setup as production-ready; call it a full-stack portfolio/demo implementation with clear hardening opportunities.

11. Key Terms

- **DTO:** Data Transfer Object; API request/response model separate from JPA entities.
- **JPA/Hibernate:** Persistence API and its ORM implementation mapping Java objects to relational tables.
- **Repository:** persistence abstraction, commonly backed by Spring Data query generation.
- **JWT:** signed token carrying claims, sent as a bearer credential; a signed JWT is not encrypted by default.
- **RBAC:** role-based access control, used here to distinguish `ROLE_USER` and `ROLE_ADMIN`.
- **Transaction:** a database unit of work expected to commit or roll back atomically.
- **Soft delete:** marking a record inactive instead of physically deleting it, preserving references/history.
- **Idempotency:** repeated submission of the same operation produces one logical outcome, important for payment/order retries.
- **CORS:** browser policy governing which web origins may call an API.

12. Source-of-Truth Index

For deeper study, open these files in this order:

- `src/App.tsx` and `src/context/` for frontend composition and state.
- `src/services/api.ts`, then `productService.ts`, `cartService.ts`, `orderService.ts`, and `authService.ts` for the API/local simulation boundary.
- `src/pages/CheckoutPage.tsx` and `src/services/razorpay.ts` for the actual browser payment flow.
- `backend/.../controller/OrderController.java`, `service/OrderService.java`, and `entity/Order.java` for server checkout and persistence.
- `backend/.../config/SecurityConfig.java`, `security/JwtAuthenticationFilter.java`, `security/JwtService.java`, `service/AuthService.java`, and `dto/RegisterRequest.java` for identity and authorization.
- `backend/.../entity/`, `repository/`, `dto/`, and `database/schema.sql` for the data model and database constraints.
- `backend/pom.xml`, `application.yml`, `Dockerfile`, and `docker-compose.yml` for build and runtime topology.

Bottom line: StyleCart is a useful interview project because it touches real full-stack concerns: stateful commerce workflows over a stateless API, relational modeling, auth, transactions, admin authorization, deployment, and external payment integration. Its strongest interview story is not “everything is production-grade”; it is that you understand the design, can trace each workflow end to end, and can identify the concrete steps required to make the demo trustworthy under real users and concurrent traffic.